

PowerGym

Relatório do Projeto de Bases de Dados e Aplicação Web em Django

Curso: Bases de Dados / Bases de Dados e Análise de Informação

Contexto: Sistema de gestão de ginásio com landing page pública, dashboard de administração, área de sócio e área de treinador

Domínio: powergym.site | Deploy: Render | Base de dados: PostgreSQL/Neon

Resumo executivo

O PowerGym é uma aplicação web desenvolvida em Django para gerir o funcionamento de um ginásio. O sistema integra uma componente pública de apresentação, autenticação de utilizadores, gestão administrativa, perfis de sócio e treinador, aulas, inscrições, planos de treino, exercícios e pagamentos. O projeto foi desenhado para cumprir os requisitos do enunciado, nomeadamente uma base de dados relacional com mais de cinco entidades, relações 1:N, formulários próprios, templates Django, administração via Django Admin, menus de navegação e uma página index/dashboard com contagens das principais tabelas.

Índice

1. 1. Introdução e enquadramento
2. 2. Objetivos do projeto
3. 3. Requisitos funcionais
4. 4. Arquitetura da aplicação
5. 5. Modelo de dados e decisões de base de dados
6. 6. Diagramas ER e físico
7. 7. Funcionalidades implementadas
8. 8. Segurança, autenticação e permissões
9. 9. Interface, usabilidade e landing page
10. 10. Deploy, base de dados em produção e ficheiros estáticos
11. 11. Testes e validação
12. 12. Limitações e trabalho futuro
13. 13. Distribuição de trabalho e expectativa de nota
14. 14. Conclusão

1. Introdução e enquadramento

O presente relatório descreve o desenvolvimento do PowerGym, uma aplicação web para gestão de ginásio construída com Django e base de dados relacional. A escolha deste contexto foi motivada por ser um cenário realista e suficientemente rico para demonstrar modelação de dados, relações entre entidades, operações CRUD, autenticação, permissões, formulários, templates e integração com serviços externos.

A aplicação foi evoluindo desde uma estrutura inicial de CRUDs simples até uma solução mais completa: primeiro foram criados os modelos centrais e as páginas administrativas; depois foram acrescentadas funcionalidades específicas do domínio, como inscrição de sócios em aulas, criação de planos de treino, gestão de pagamentos e perfis personalizados. Na fase final, o projeto foi preparado para produção com deploy no Render, base de dados PostgreSQL no Neon, armazenamento de imagens com Cloudinary e domínio próprio powergym.site.

2. Objetivos do projeto

- Criar uma aplicação Django funcional e executável, com código organizado por modelos, views, forms, urls e templates.
- Construir uma base de dados relacional com pelo menos cinco entidades próprias, excluindo tabelas de ligação, e com relações 1:N.
- Disponibilizar CRUDs completos para as entidades principais através de páginas desenvolvidas pelo grupo.
- Permitir a gestão das mesmas entidades também através do Django Admin.
- Criar uma página index/dashboard com ligações e contagens das principais tabelas.
- Evoluir a aplicação para um produto mais profissional, com landing page pública, áreas autenticadas e design mais cuidado.
- Colocar a aplicação online com base de dados PostgreSQL, domínio próprio e armazenamento externo para imagens.

3. Requisitos funcionais

ID	Requisito	Descrição
RF01	Landing page pública	O visitante deve conseguir aceder a uma página inicial pública com apresentação do PowerGym, modalidades em destaque, equipa e botões para login ou registo.
RF02	Registo de sócio	Um visitante deve conseguir criar uma conta de sócio, indicando dados pessoais, credenciais e opcionalmente uma fotografia de perfil.
RF03	Autenticação	O sistema deve permitir login e logout, redirecionando utilizadores para a área adequada conforme o tipo de conta.
RF04	Dashboard de administração	O administrador deve aceder a um painel com estatísticas do ginásio, incluindo contagens de sócios, treinadores, aulas, modalidades, inscrições, planos, exercícios e pagamentos.
RF05	Gestão de sócios	O administrador deve poder listar, consultar, criar, editar e apagar sócios.
RF06	Gestão de treinadores	O administrador deve poder gerir treinadores, especialidades, biografias, fotografias e contas associadas.

RF07	Gestão de modalidades e aulas	O administrador deve poder gerir modalidades e criar aulas com data, hora, lotação, treinador e modalidade.
RF08	Inscrição em aulas	O sistema deve permitir inscrever sócios em aulas, evitando inscrições duplicadas na mesma aula.
RF09	Área de sócio	O sócio deve consultar dados pessoais, passe digital por QR Code, aulas inscritas, pagamentos e plano de treino.
RF10	Área de treinador	O treinador deve consultar as aulas associadas ao seu perfil.
RF11	Planos de treino e exercícios	O administrador deve criar planos de treino por sócio e treinador, e associar exercícios com séries, repetições e descanso.
RF12	Gestão de pagamentos	O administrador deve criar e controlar pagamentos, estados pendentes, liquidados e atrasados, incluindo mensalidades.
RF13	Gestão de acessos	O administrador deve criar contas de administrador, treinador ou associar contas a sócios/treinadores existentes.
RF14	Proteção de páginas	As páginas administrativas devem estar protegidas, sendo acessíveis apenas a administradores autenticados.

4. Arquitetura da aplicação

A aplicação segue a arquitetura habitual de um projeto Django. O projeto principal chama-se `ginasio_project` e contém as configurações, urls globais e ficheiros WSGI/ASGI. A aplicação de domínio chama-se `ginasio` e concentra a lógica do ginásio: modelos, formulários, views, urls, templates e configuração do admin.

Componente	Ficheiros principais	Responsabilidade
Projeto Django	<code>ginasio_project/settings.py</code> , <code>urls.py</code> , <code>wsgi.py</code>	Configuração global, base de dados, static/media, middleware, deploy e roteamento principal.
Aplicação ginasio	<code>models.py</code> , <code>views.py</code> , <code>forms.py</code> , <code>urls.py</code>	Implementação do domínio PowerGym e das regras funcionais.
Templates	<code>ginasio/templates/ginasio/*.html</code>	Interface HTML das páginas públicas, administrativas e perfis.
Estáticos	<code>static/css/style.css</code> , <code>static/images/*</code>	Estilos globais, logo, favicons e recursos visuais.
Admin	<code>ginasio/admin.py</code>	Registo das entidades no Django Admin com filtros e pesquisa.
Migrations	<code>ginasio/migrations/*.py</code>	Histórico de evolução do modelo físico da base de dados.

5. Modelo de dados e decisões de base de dados

O modelo de dados foi desenhado para representar o funcionamento de um ginásio real. Em vez de limitar a aplicação a uma lista isolada de membros, foram criadas entidades ligadas entre si: sócios participam em

aulas, aulas pertencem a modalidades e são dadas por treinadores, sócios têm pagamentos e planos de treino, e planos de treino contêm exercícios.

A base de dados em produção usa PostgreSQL através do Neon. Durante o desenvolvimento local foi usado SQLite como fallback, mas a configuração final lê DATABASE_URL através de variáveis de ambiente, permitindo que o Render se ligue à base de dados Neon de forma segura.

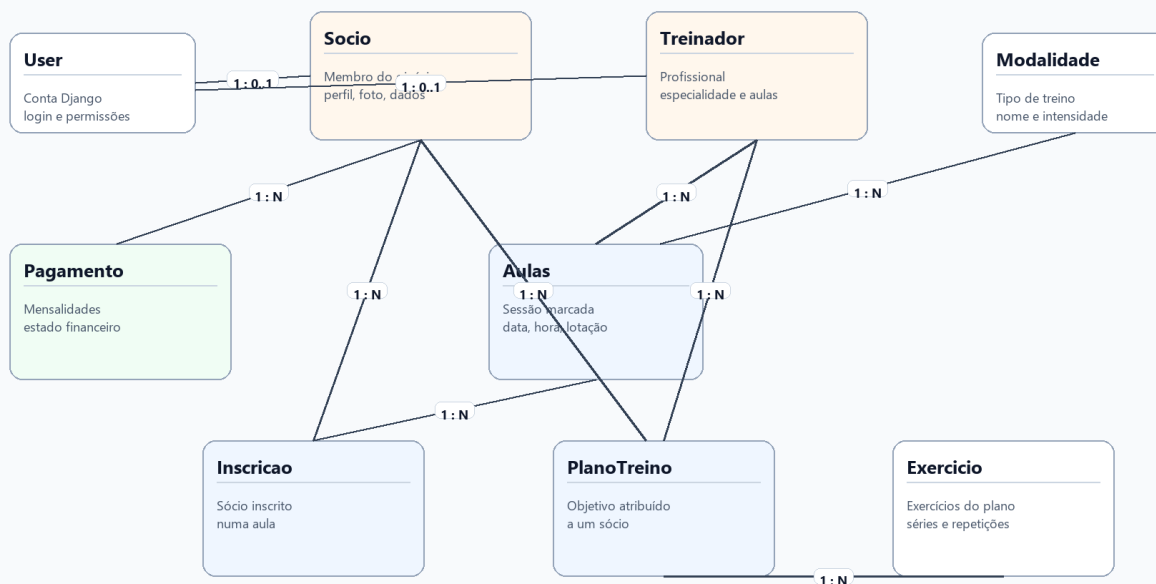
Entidade	Finalidade	Relações principais
Socio	Representa um membro do ginásio.	Pode ter vários pagamentos, inscrições e planos de treino. Pode estar associado a auth_user.
Treinador	Representa um profissional do ginásio.	Pode lecionar várias aulas e criar vários planos de treino. Pode estar associado a auth_user.
Modalidade	Representa um tipo de treino, como musculação, funcional ou natação.	Pode estar associada a várias aulas.
Aulas	Representa uma sessão marcada no calendário.	Pertence a uma modalidade e a um treinador; pode ter várias inscrições.
Inscricao	Liga um sócio a uma aula.	É a entidade associativa entre Socio e Aulas, com data de registo.
PlanoTreino	Representa o plano atribuído a um sócio.	Pertence a um sócio e a um treinador; contém vários exercícios.
Exercicio	Representa um exercício dentro de um plano.	Pertence a um plano de treino.
Pagamento	Representa mensalidades ou pagamentos únicos.	Pertence a um sócio e guarda valor, datas, estado e frequência.

6. Diagramas ER e físico

O diagrama ER apresenta a visão conceptual das entidades e relações. O modelo físico mostra a tradução para tabelas e campos concretos usados pela aplicação Django e pela base de dados PostgreSQL.

Modelo Entidade-Relacionamento - PowerGym

Entidades principais e cardinalidades funcionais do sistema.



Nota: auth_user é a tabela de utilizadores do Django; Socio e Treinador podem estar associados a contas de autenticação.

Figura 1 - Diagrama entidade-relacionamento do PowerGym.

Modelo Físico da Base de Dados

Tabelas, chaves primárias e chaves estrangeiras implementadas em Django/PostgreSQL.

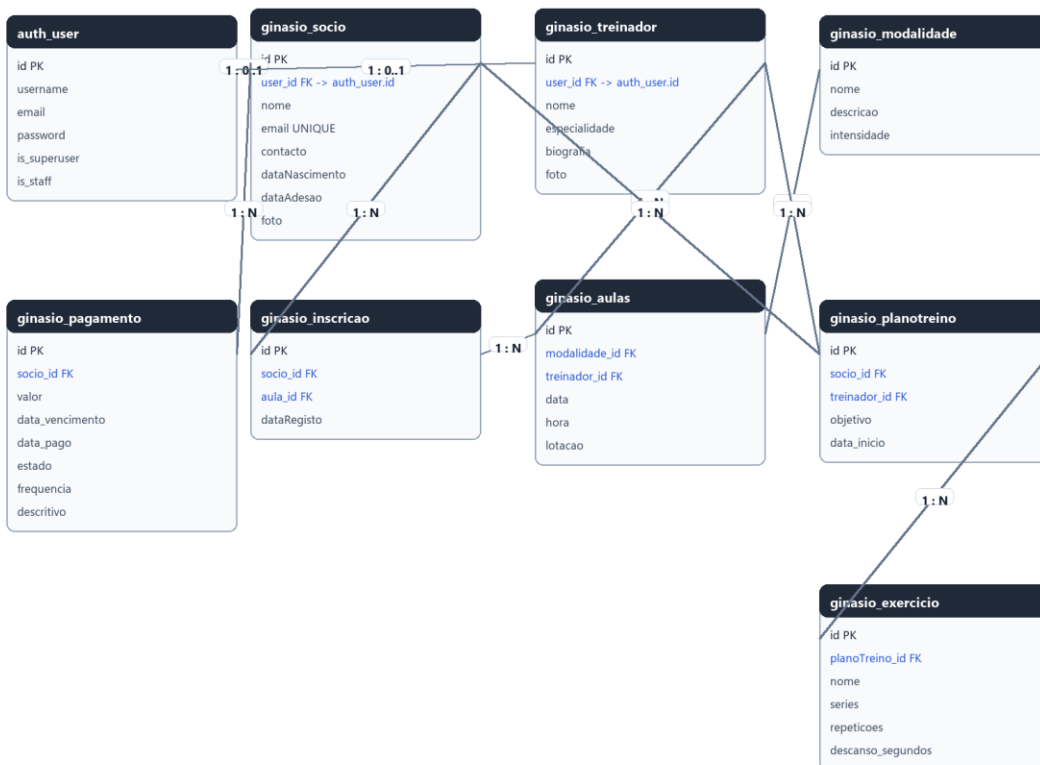


Figura 2 - Modelo físico simplificado das tabelas principais.

7. Funcionalidades implementadas

7.1 Landing page pública

A landing page foi reformulada para funcionar como entrada pública do PowerGym. Mostra o nome da marca, uma mensagem de valor, botões para começar ou fazer login, estatísticas vindas da base de dados, modalidades, equipa e uma secção de chamada para a área de sócio. A página usa dados reais das tabelas Modalidade, Treinador, Socio e Aulas.

7.2 Dashboard de administração

O dashboard resume o estado geral do ginásio. Para além das contagens principais, inclui gráficos com Chart.js: um gráfico circular para estados financeiros e um gráfico de barras para volume de dados. Esta página cumpre o requisito de existir uma index/dashboard com links e contagens das tabelas.

7.3 CRUDs administrativos

Foram implementadas páginas de listagem, detalhe, criação, edição e eliminação para as entidades principais. Os formulários usam ModelForm, reduzindo duplicação entre o modelo e a interface. As páginas de confirmação de eliminação reduzem o risco de apagar dados por engano.

7.4 Área de sócio

Após login, um sócio acede à sua área pessoal. Esta página mostra dados pessoais, QR Code de acesso, aulas em que está inscrito, pagamentos e plano de treino. O QR Code é gerado dinamicamente com a biblioteca qrcode e convertido para Base64 para ser apresentado diretamente no template.

7.5 Área de treinador

A área de treinador permite que um profissional veja as aulas associadas ao seu perfil. A associação é feita através do campo user em Treinador, permitindo distinguir contas de sócio, treinador e administrador.

7.6 Pagamentos e mensalidades

O sistema gere pagamentos com estados Liquidado, Pendente e Atrasado. As views atualizam pagamentos pendentes para Atrasado quando a data de vencimento já passou. Quando uma mensalidade é liquidada, a lógica cria a mensalidade seguinte, automatizando parte do processo financeiro.

8. Segurança, autenticação e permissões

A autenticação usa o sistema padrão do Django. A view redirecionar_login decide para onde cada utilizador deve ir depois de entrar: administrador para o dashboard, treinador para a área de treinador e sócio para a área de cliente.

- As páginas administrativas usam um decorador `admin_required`, que bloqueia acesso a visitantes, sócios e treinadores.
- O sócio pode cancelar apenas as suas próprias inscrições.
- As credenciais de produção foram removidas do código e passaram a variáveis de ambiente no Render.
- DEBUG deve estar False em produção.
- SECRET_KEY, DATABASE_URL e credenciais Cloudinary são configuradas fora do repositório.

9. Interface, usabilidade e landing page

A interface foi melhorada em duas fases. Primeiro, a landing page foi redesenhada para causar boa primeira impressão e explicar rapidamente a proposta do PowerGym. Depois, as páginas internas foram harmonizadas com uma navbar escura, botões consistentes, cards, tabelas e formulários com aspeto mais profissional.

Também foi acrescentado feedback de erro no login. Quando um utilizador tenta entrar com nome ou palavra-passe incorretos, aparece uma mensagem clara em vermelho. Esta decisão melhora a experiência do utilizador porque evita uma falha silenciosa.

10. Deploy, base de dados em produção e ficheiros estáticos

O projeto foi colocado online no Render e ligado ao domínio `powergym.site`. A base de dados de produção encontra-se no Neon, usando PostgreSQL. As imagens enviadas pelos utilizadores são guardadas no Cloudinary através de `django-cloudinary-storage`. Os ficheiros estáticos, como CSS, logo e favicons, são servidos com apoio de WhiteNoise.

Serviço	Uso no projeto	Justificação
---------	----------------	--------------

Render	Alojamento da aplicação Django.	Permite deploy automático a partir do GitHub e execução com Gunicorn.
Neon	Base de dados PostgreSQL.	Cumprir o requisito de usar PostgreSQL e separar os dados do servidor da aplicação.
Cloudinary	Armazenamento de imagens de sócios/treinadores.	Evita depender do disco efêmero do Render para ficheiros enviados.
WhiteNoise	Serviço de ficheiros estáticos.	Permite servir CSS, logo e recursos estáticos em produção.
GitHub	Controlo de versões e integração com deploy.	Regista evolução do projeto e facilita publicação no Render.

11. Testes e validação

Foram realizados testes manuais e verificações automáticas simples durante a fase final. O objetivo foi garantir que as páginas principais carregavam, que o login inválido apresentava mensagem, que o dashboard funcionava, que as páginas administrativas estavam protegidas e que não existiam migrations pendentes.

- `python manage.py check` sem erros.
- `python manage.py makemigrations --check --dry-run` sem alterações pendentes.
- Validação de landing page, login, dashboard e rotas principais.
- Correção de erro 500 na página de apagar plano de treino.
- Correção de permissões para impedir acesso público às páginas de gestão.
- Correção de ficheiros estáticos e logo em produção.

12. Limitações e trabalho futuro

- Criar testes automatizados permanentes para views e formulários.
- Adicionar validação de lotação das aulas, impedindo inscrições acima da capacidade.
- Melhorar relatórios financeiros com filtros por mês e exportação.
- Permitir que treinadores editem planos dos seus próprios sócios, caso se pretenda esse fluxo.
- Adicionar recuperação de password por email.
- Criar histórico de check-ins através do QR Code.

13. Distribuição de trabalho e expectativa de nota

Esta secção deve ser preenchida pelo grupo antes da entrega, conforme pedido no enunciado. A tabela abaixo foi deixada preparada para indicar percentagem de trabalho e expectativa de nota individual.

Elemento	% de trabalho	Principais contributos	Expectativa de nota
Elemento 1	A preencher	A preencher	A preencher
Elemento 2	A preencher	A preencher	A preencher

14. Conclusão

O PowerGym cumpre os objetivos centrais do projeto: é uma aplicação Django funcional, com base de dados relacional, mais de cinco entidades relacionadas, relações 1:N, CRUDs próprios, Django Admin, dashboard com contagens, autenticação, permissões e deploy em produção. Para além dos requisitos mínimos, a aplicação foi evoluída para uma experiência mais profissional, com landing page pública, áreas diferenciadas por tipo de utilizador, gráficos, QR Code, Cloudinary, Neon, Render e domínio próprio.

A modelação da base de dados foi pensada para representar um cenário real de ginásio e não apenas um exemplo simplificado. As relações entre sócios, aulas, inscrições, treinadores, planos, exercícios e pagamentos demonstram o uso prático de chaves estrangeiras, integridade relacional e lógica de domínio.

Apêndice A - Mapeamento de entidades para ficheiros Django

Entidade	Model	Form	Views/URLs	Templates
Socio	Socio	SocioForm	socio_list/detail/create/update/delete	socio_*.html
Treinador	Treinador	TreinadorForm	treinador_list/detail/create/update/delete	treinador_*.html
Modalidade	Modalidade	ModalidadeForm	modalidade_list/detail/create/update/delete	modalidade_*.html
Aulas	Aulas	AulasForm	aula_list/detail/create/update/delete	aula_*.html
Inscricao	Inscricao	InscricaoForm	inscricao_list/detail/create/update/delete	inscricao_*.html
PlanoTreino	PlanoTreino	PlanoTreinoForm	planotreino_list/detail/create/update/delete	planotreino_*.html
Exercicio	Exercicio	ExercicioForm	exercicio_list/detail/create/update/delete	exercicio_*.html
Pagamento	Pagamento	PagamentoForm	pagamento_list/detail/create/update/delete	pagamento_*.html

Apêndice B - Decisões técnicas relevantes

- Django foi usado porque o enunciado e as aulas laboratoriais se baseiam neste framework.
- ModelForm foi usado para reduzir duplicação e manter formulários ligados diretamente aos modelos.
- PostgreSQL/Neon foi usado em produção para cumprir o requisito de base de dados PostgreSQL.
- Cloudinary foi usado porque o Render não garante persistência local para ficheiros enviados.
- WhiteNoise foi usado para servir ficheiros estáticos em produção.
- Gunicorn foi usado como servidor WSGI em produção no Render.
- Variáveis de ambiente foram usadas para evitar expor segredos no código.